

Soluții explicate — victor-vectori

Companion la CODE_REVIEW.md · toate headerele din `vectors/` · 17.07.2026

Pentru fiecare constatare din review: **problema** (unde și ce se vede), **mecanismul** (de ce se întâmplă, ce face limbajul de fapt) și **soluția** (cod corect + de ce funcționează). Codul original nu a fost modificat — variantele de aici sunt referință de studiu.

● Critice

B1 — `cmmmc` returnează mereu `a` (operatorul virgulă)

`corectareTema2.h:249` · `tema2.h:655`

PROBLEMA Formula matematică $[a,b] = a \cdot b / (a,b)$ a fost copiată direct în cod:

```
return a * b / (a, b);
```

MECANISMUL În C++, `(a, b)` nu înseamnă „cmmdc de `a` și `b`”. Este **operatorul virgulă**: evaluează expresia din stânga, aruncă rezultatul și întoarce expresia din dreapta. Deci `(a, b)` valorează `b`, iar întreaga expresie devine `a * b / b`, adică `a`. Compilatorul nu protestează pentru că expresia e perfect validă sintactic — doar că are alt sens decât notația de pe hârtie. `solutie3f` afișează astfel 56 în loc de `cmmmc(56, 196) = 392`.

SOLUȚIA Cmmdc-ul trebuie *apelat*, nu notat:

```
int cmmdc(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int cmmmc(int a, int b) {
    if (a == 0 || b == 0) {
        return 0;
    }
    return a / cmmdc(a, b) * b;
}
```

Detaliu fin: am scris `a / cmmdc(a, b) * b`, nu `a * b / cmmdc(a, b)`. Împărțirea întâi ține rezultatul intermediar mic — `a * b` poate depăși `int` pentru numere mari, iar `a / cmmdc(a, b)` e mereu exact (cmmdc divide `a`).

B2 — comparația înlănțuită de parități

corectareTema2.h:110

PROBLEMA Verificarea „toate trei au aceeași paritate”:

```
if (v[i] % 2 == v[i - 1] % 2 == v[i + 1] % 2)
```

MECANISMUL C++ evaluează stânga→dreapta: întâi `v[i]%2 == v[i-1]%2`, care produce un `bool` (0 sau 1); apoi acel 0/1 se compară cu `v[i+1]%2`. Pentru trei numere pare: $(0 == 0) \rightarrow 1$, apoi $1 == 0 \rightarrow \text{false}$. Exact cazul căutat pică testul. Matematica permite lanțul `a = b = c`; C++ nu — fiecare egalitate e o operație separată care produce `bool`.

SOLUȚIA Două comparații legate cu `&&`:

```
if (v[i] % 2 == v[i - 1] % 2 && v[i - 1] % 2 == v[i + 1] % 2)
```

B3 — acces în afara vectorului: `v[-1]`

corectareTema2.h:109

PROBLEMA Verificarea vecinilor se face pe *valoare*:

```
if (v[i + 1] != 0 && v[i - 1] >= 1)
```

MECANISMUL La prima iterație `i = 0`, deci `v[i-1]` este `v[-1]` — o citire *înainte* de începutul vectorului. Este comportament nedefinit: poate merge azi, poate crăpa mâine, poate da valori aleatorii. Întrebarea „are elementul un vecin?” este o întrebare despre **index** (poziție), nu despre valoarea stocată acolo — valoarea nu-ți poate spune dacă poziția există.

SOLUȚIA Granițele se verifică pe index:

```
for (int i = 0; i < d; i++) {
    if (i > 0 && i < d - 1) {
        if (patratPerfect(v[i]) == false
            && v[i] % 2 == v[i - 1] % 2
            && v[i - 1] % 2 == v[i + 1] % 2) {
            afisareDescompunereFactoriPrimi(v[i]);
        }
    }
}
```

`i > 0` garantează că există vecin în stânga, `i < d - 1` că există în dreapta. Primul și ultimul element pur și simplu nu au ambii vecini — deci nu pot îndeplini condiția.

B4 — cmmdc prin scăderi: buclă infinită pentru 0

realizareaUnuiNouVector.h:59-70 · declanșat în solutie3a (:533-544)

PROBLEMA Varianta cu scăderi repetate:

```
int cmmdc(int a, int b) {
    while (a != b) {
        if (a > b) {
            a = a - b;
        }
        if (b > a) {
            b = b - a;
        }
    }
    return a;
}
```

MECANISMUL Dacă unul dintre numere e 0, scăderea nu mai schimbă nimic: `cmmdc(0, 45) → b = 45 - 0 = 45`, la infinit. Programul „îngheață” fără nicio eroare. Și nu e un caz teoretic: în `solutie3a` vectorul `{-15, -5, 0, 35, 45, 25}` conține 0, funcția `minim` întoarce 0, iar apelul `cmmdc(0, 45)` blochează programul. Varianta cu scăderi cere implicit `a > 0 && b > 0` — o precondiție pe care nimeni n-o verifică.

SOLUȚIA Euclid cu rest (deja scris corect în `dublete.h:18`):

```
int cmmdc(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

De ce nu se blochează: restul `a % b` e *strict mai mic* decât `b`, deci `b` scade la fiecare pas și ajunge garantat la 0. Iar `cmmdc(0, 45)` intră o singură dată în buclă și întoarce 45 — matematic corect, `cmmdc(0, n) = n`.

B5 — dimensiune falsă + dublet cu element inexistent

dublete.h:29-41

PROBLEMA

```
int v[100] = { 5 };
int d = 5;
for (int i = 0; i < d; i++) {
    if (cmmdc(v[i], v[i + 1]) == 1) {
```

MECANISMUL Două defecte suprapuse. (1) Inițializarea `{ 5 }` pune 5 pe poziția 0 și **zero** pe restul — deci `d = 5` declară patru elemente care nu există în date. (2) Bucla merge până la `i = d - 1 = 4` și acolo citește `v[5]` — perechea (4, 5) folosește un element din afara datelor. Pentru dublete (perechi alăturate), ultima pereche validă e (d-2, d-1), deci bucla se oprește la `i < d - 1`.

SOLUȚIA

```
int v[100] = { 5, 8, 9, 14, 25 };
int d = 5;
int ct = 0;

for (int i = 0; i < d - 1; i++) {
    if (cmmdc(v[i], v[i + 1]) == 1) {
        ct++;
    }
}
cout << ct;
```

Regula de disciplină: `d` este contractul dintre date și cod — spune exact câte valori sunt reale. Orice buclă care atinge `v[i + 1]` trebuie să se oprească cu un pas mai devreme.

B6 — același off-by-one în soluție2b

realizareaUnuiNouVector.h:364-365

PROBLEMA `for (i = 0; i < d; i++)` cu `nrCifre(v[i + 1])` — la `i = 4` se compară cu `v[5]`, care nu face parte din date.

MECANISMUL Aici e mai periculos decât la B5: `v[5]` e 0 (completare automată), `nrCifre(0)` întoarce 0, comparația dă `false`, și programul *pare* să meargă corect. Bug-urile silențioase sunt cele mai scumpe — nu ai crash care să te alerteze, doar un rezultat subtil greșit când datele se schimbă.

SOLUȚIA

```
for (int i = 0; i < d - 1; i++) {
    if (nrCifre(v[i]) == nrCifre(v[i + 1])) {
        cout << v[i] << "," << v[i + 1] << endl;
    }
}
```

B7 — sortezi `aux`, afișezi `v`

sortareaVectorilor.h:108-109

PROBLEMA

```
sortatPrinBubbleSort(aux, ct);
afisare(v, d);
```

MECANISMUL Toată munca — filtrarea numerelor prime în `aux`, sortarea lor — se face corect, apoi la final se afișează vectorul original neatins. Rezultatul rulării arată ca și cum programul n-ar face nimic. Un test simplu l-ar fi prins imediat: output-ul așteptat era `7 29 53 113`, cel real e vectorul inițial.

SOLUȚIA

```
sortatPrinBubbleSort(aux, ct);
afisare(aux, ct);
```

Atenție și la dimensiune: `aux` are `ct` elemente valide, nu `d`.

B8 — `d = 8` pe un vector cu 6 elemente

tema3.h:635-636

PROBLEMA

```
int v[100] = { 8, 9, 9, 4, 5, 7 };  
int d = 8;
```

MECANISMUL Pozițiile 6 și 7 sunt 0 (completare automată la inițializarea parțială) și intră în sortare ca elemente reale — output-ul conține două zerouri care nu există în date. Același contract ca la B5: `d` trebuie să fie exact numărul de valori scrise în acoladă.

SOLUȚIA

```
int v[100] = { 8, 9, 9, 4, 5, 7 };  
int d = 6;
```

● Importante

M1 — funcții definite duplicat în headere

prim × 6 fișiere · afisare × 5 · cmmdc, palindrom, baza2, cifControl, solutiela...

PROBLEMA Aceleași funcții sunt *definite* (cu corp) în mai multe headere. Azi merge doar pentru că `vectori.cpp` include un singur header. La primul include dublu:

```
#include "dublete.h"
#include "tema2.h"
```

```
error C2084: function 'bool prim(int)' already has a body
```

MECANISMUL `#pragma once` previne includerea *aceluiași fișier* de două ori — nu are cum să împiedice două fișiere *diferite* să definească aceeași funcție. Regula limbajului (One Definition Rule): o funcție are voie la o singură definiție per program. Headerul e locul pentru *declarații*; definițiile stau în `.cpp`.

SOLUȚIA Pragmatic pentru structura actuală: un singur `utile.h` cu funcțiile comune, iar headerele de teme doar îl includ:

```

#pragma once
#include <iostream>
using namespace std;

bool prim(int n) {
    if (n <= 1) {
        return false;
    }
    for (int k = 2; k <= n / 2; k++) {
        if (n % k == 0) {
            return false;
        }
    }
    return true;
}

int cmmdc(int a, int b) {
    while (b != 0) {
        int r = a % b;
        a = b;
        b = r;
    }
    return a;
}

void afisare(int v[], int d) {
    for (int i = 0; i < d; i++) {
        cout << v[i] << " ";
    }
    cout << endl;
}

```

Apoi în fiecare temă: `#include "utile.h"` și se șterg copiile locale. Aici `#pragma once` chiar ajută — dacă două teme includ amândouă `utile.h`, fișierul intră o singură dată. Rămâne un pas: funcțiile `solutieXY` cu același nume între teme trebuie redenumite distinct (de exemplu `tema2_solutie1a`) dacă vrei să incluzi mai multe teme simultan.

M2 — poziția -1 nefiltrată în MunteVale

MunteVale.h:119–126

PROBLEMA `deterPozitie1Ex1` întoarce -1 când nu găsește al 4-lea element potrivit, iar apelantul folosește valoarea direct: `sortatDescrescatorInterval(v, -1, d - 1)` pornește bucla de la `i = -1` → `v[-1]`.

MECANISMUL -1 e o convenție bună de semnalizare („nu am găsit”), dar convenția funcționează doar dacă *apelantul o verifică*. Cu datele actuale poziția găsită e 4, deci bug-ul e invizibil — până schimbi vectorul.

SOLUȚIA

```
int poz = deterPozitie1Ex1(v, d);
if (poz == -1) {
    cout << "Nu exista al 4-lea element cu 3-5 divizori proprii" << endl;
    return;
}
sortatCrescatorInterval(v, 0, poz);
sortatDescrescatorInterval(v, poz, d - 1);
afisare(v, d);
```

M3 — 6 valori, d = 5

realizareaUnuiNouVector.h:418–419

PROBLEMA `{ 4321,1234,5414,6531,6425,5246 }` cu `d = 5`. Combinația `i < d + v[i+1]` atinge accidental și al 6-lea element, deci „merge” — din motivul greșit.

SOLUȚIA `d = 6` și bucla `i < d - 1` — aceeași regulă ca la B5/B6/B8: dimensiunea reflectă datele, iar accesul la `i + 1` scurtează bucla cu un pas.

M4 — baza2 promite long long, calculează în int

tema2.h:230-241

PROBLEMA

```
long long baza2(int n) {  
    int b2 = 0;  
    int p = 1;
```

MECANISMUL Reprezentarea binară „scrisă în zecimal” crește exploziv: $n = 1024$ devine 10000000000 (11 cifre), peste maximul `int` (~2.1 miliarde). Depășirea se produce în `b2` și `p`, *înainte* de return — tipul de retur `long long` nu mai poate repara o valoare deja trunchiată. Tipul mare trebuie să fie acolo unde se *acumulează*, nu doar la ieșire.

SOLUȚIA

```
long long baza2(int n) {  
    long long b2 = 0;  
    long long p = 1;  
  
    while (n > 0) {  
        int r = n % 2;  
        b2 = r * p + b2;  
        p = p * 10;  
        n = n / 2;  
    }  
    return b2;  
}
```

M5 — testul de impar % 2 == 1 pică pe negative

exercitiiVectori.h:36

MECANISMUL În C++, restul păstrează semnul primului operand: `-7 % 2` este `-1`, nu `1`. Deci `v[i] % 2 == 1` ratează toate imparele negative.

SOLUȚIA Testează „nu e par” în loc de „restul e exact 1”:

```
if (v[i] % 2 != 0) {  
    ctImp++;  
}
```

`-7 % 2 = -1 ≠ 0` → numărat corect. Idiomul `!= 0` e robust pentru orice semn.

M6 — santinele magice 9999 / -9999

realizareaUnuiNouVector.h:505, 520, 763, 776

MECANISMUL `int min = 9999` funcționează doar dacă vectorul conține ceva sub 9999. Cu `{10000, 20000}`, minimul „găsit” e 9999 — un număr care nu există în vector. Orice constantă-santinelă e un pariu pe datele de intrare.

SOLUȚIA Pornește de la primul element — nu are cum să dea greș:

```
int minim(int v[], int d) {
    int min = v[0];

    for (int i = 1; i < d; i++) {
        if (v[i] < min) {
            min = v[i];
        }
    }
    return min;
}
```

M7 — solutie3b din tema2 e neterminată

tema2.h:523–535

PROBLEMA Corpul condiției conține doar `int desc = 0;` cu un comentariu-întrebare. Descompunerea în factori primi există deja, scrisă corect: `afisareDescompunereFactoriPrimi` în `corectareTema2.h:79`.

SOLUȚIA Refolosire, nu rescriere:

```
for (int i = 0; i < d - 1; i++) {
    if (patratPerfect(v[i]) == false && v[i] % 2 == v[i + 1] % 2) {
        afisareDescompunereFactoriPrimi(v[i]);
    }
}
```

Observă și corecția de graniță: bucla merge până la `d - 1` pentru că folosește `v[i + 1]` — condiția `i != d - 1` din interior devine inutilă.

Cleanups

C1 — bubble sort rescris de ~8 ori

tema3.h:8, 182, 356, 460, 494, 550...

Toate variantele au schelet identic — diferă doar *criteriul* din `if`. Când observi că ai copiat aceeași funcție și ai schimbat un rând, acela e semnalul pentru refolosire. La nivelul actual: păstrează o singură pereche crescător/descrescător și, unde criteriul diferă (suma cifrelor, prima cifră), calculează criteriul într-un vector separat și sortează-l pe acela împreună cu originalul — exact pattern-ul pe care l-ai folosit deja corect la `sortatPrinBubbleSortInaltime` cu vectorul de poziții.

C2 — `if (cond) return true; return false;`

tema2.h:117–121 · realizareaUnuiNouVector.h:141–144 · corectareTema2.h:31–34 ...

Condiția este *deja* valoarea bool pe care o vrei:

```
bool primCifEgalCuUltimCif(int n) {
    int cif = n % 10;
    while (n > 9) {
        n = n / 10;
    }
    return cif == n;
}
```

C3 — bool cu 0/1 și `== false`

tema2.h:176 · corectareTema2.h:51 · `while (sortat == false)` peste tot

`bool vf = 1` → `bool vf = true`; `if (vf == 1)` → `if (vf)`; `while (sortat == false)` → `while (!sortat)`. Tipul bool are propriile literale — folosirea lor spune cititorului imediat că variabila e un steag, nu un număr.

C4 — nume care mint

tema3.h:282 · sortareaVectorilor.h:19 · tema2.h:108

`bubbleSortPatratPerfectCrescator` și `sortareaPrinSelectie` fac de fapt exchange sort (swap direct între pozițiile `i` și `j`) — bubble compară doar *vecini*, selection face *un singur swap* per iterație externă. Numele greșit al algoritmului te va încurca la examen. `primCifEgalUltimCif` — capitalizare inconsecventă (`primCifEgalUltimCif`).

C5 — cod mort

tema2.h:651 · MunteVale.h:132–155 · tema2.h:641 · corectareTema2.h:234 ·
realizareaUnuiNouVector.h:375 · exercitiiVectori.h:49

`cmmmc` din tema2 nu e apelat nicăieri; `baza2 / ctCifre1` la finalul MunteVale la fel; `int poz = i + 1` declarat și nefolosit în două locuri; `int aux` nefolosit în `ogl`; un `;;` dublu. Codul mort nu strică rularea, dar fiecare rând nefolosit e un rând pe care cititorul îl analizează degeaba.

C6 — `patratPerfect` cu comparație pe float

tema3.h:278 · corectareTema2.h:66

`sqrt(n) == (int)sqrt(n)` compară numere în virgulă mobilă — fragil la valori mari, unde `sqrt` poate întoarce 99.9999... pentru 10000. Varianta robustă (pe care o ai deja în tema2.h:514):

```
bool patratPerfect(int n) {  
    int radacina = (int)round(sqrt(n));  
    return radacina * radacina == n;  
}
```

Verificarea finală se face înapoi în întregi — acolo egalitatea e exactă.

C7 — `min / max` ca nume proprii sub `using namespace std`

realizareaUnuiNouVector.h:762, 775

Coliziune cu `std::min / std::max` — merge acum pentru că semnăturile diferă, dar e o sursă clasică de erori criptice de compilare. Numele `minim / maxim` folosite în prima variantă erau alegerea mai sigură. În plus, amestecul `long long int minim` cu vector de `int` nu aduce nimic — toate valorile încap în `int`.

Firul roșu al review-ului: mecanica algoritmilor e stăpânită (trace-urile pe hârtie din comentarii o dovedesc), punctele pierdute sunt sistematic la **granițe** — `d` care nu corespunde datelor și `v[i±1]` fără verificare de index. Înainte de fiecare buclă, două întrebări: „câte elemente reale am?” și „ating vreun vecin? — atunci scurtez bucla”.